

Chap 3. 머신 러닝 핵심 알고리즘

3.1 지도 학습

Date: 2026. 03. 24

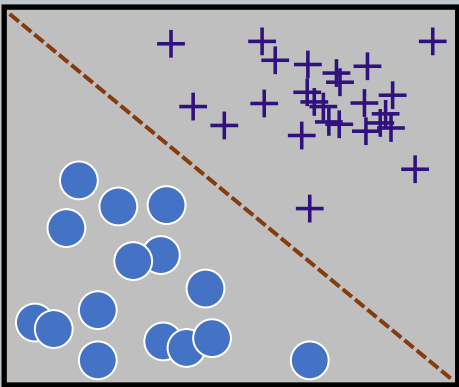
Presenter: 한인철

지도 학습 (Supervised Learning)

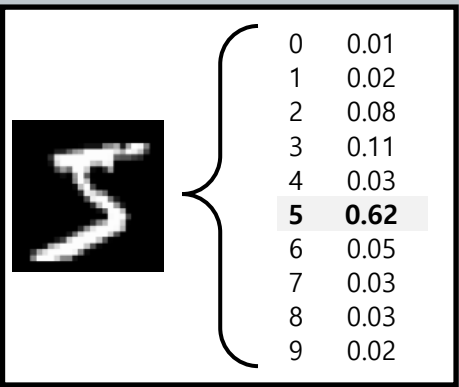
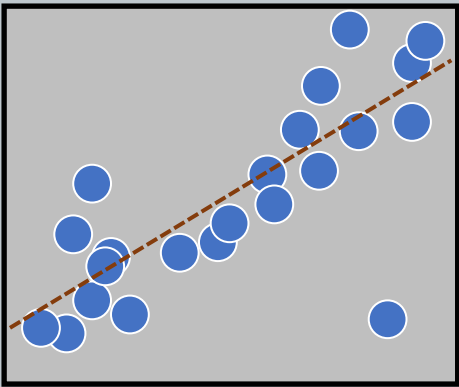
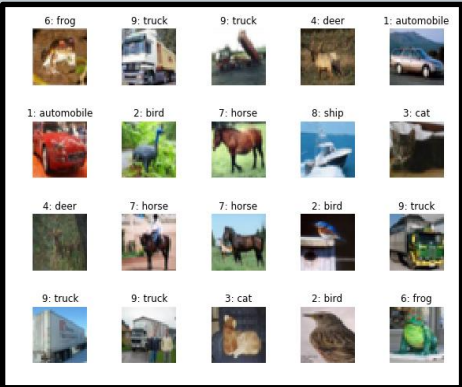
지도 학습은 정답(레이블(label) 또는 실제 값(Ground Truth))을 컴퓨터에 미리 알려 주고 데이터를 학습시키는 방법임

지도 학습에는 분류(Classification)과 회귀(Regression)가 있음

구분	분류	회귀
데이터 유형	이산형 데이터	연속형 데이터
결과	훈련 데이터의 레이블 중 하나를 예측	연속된 값을 예측
예시	학습 데이터를 $A \cdot B \cdot C$ 그룹 중 하나로 매핑	결과 값이 어떤 값이든 나올 수 있음



분류 지도 학습 예시



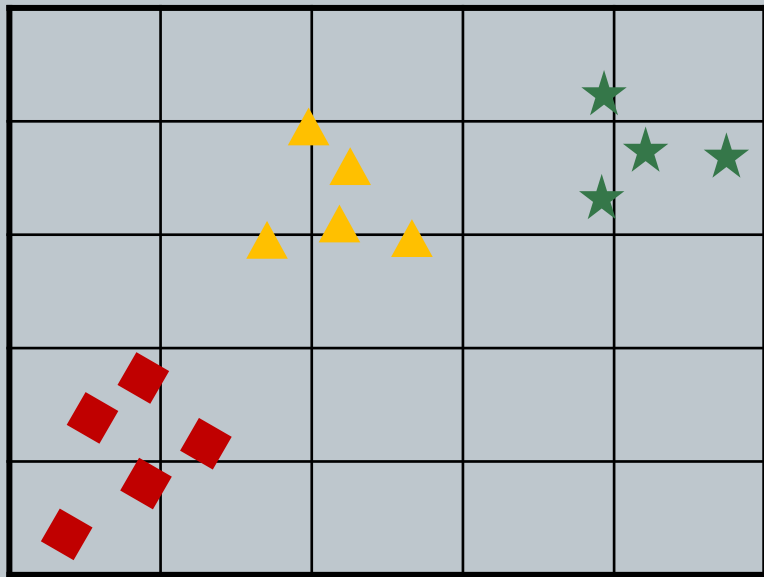
분류 지도 학습 예시

지도 학습 알고리즘: K-최근접 이웃

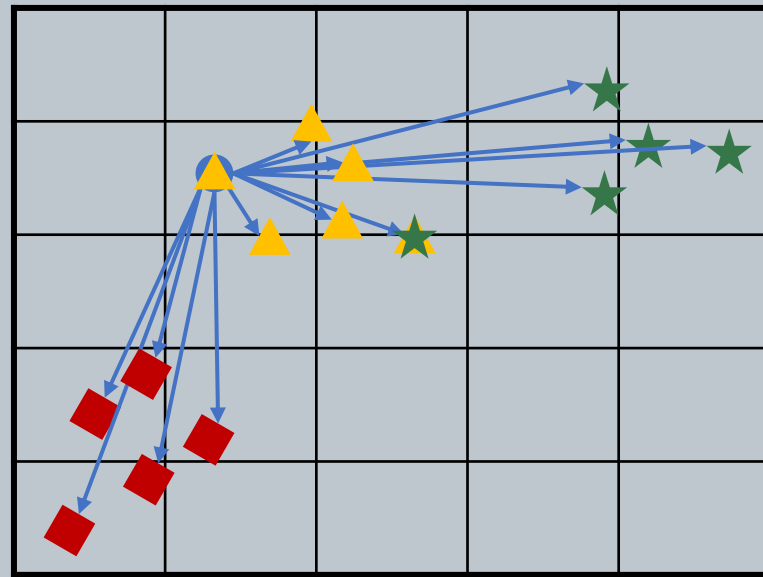
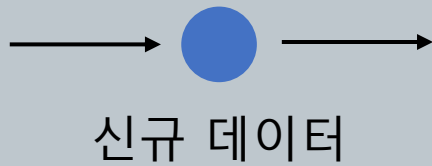
K-최근접 이웃(K-Nearest Neighbor, KNN) 알고리즘은 새로운 입력을 받을 때, 기존 클러스터에서 모든 데이터와 인스턴스 기반 거리를 측정한 후 가장 많은 속성을 가진 클러스터에 할당하는 분류 알고리즘임

과거 데이터를 저장해두고, 새로운 데이터가 추가 및 삭제될 때마다 비교를 수행하는 방식임

K: 5



기존 데이터 클러스터



새로운 데이터 클러스터

지도 학습 알고리즘: K-최근접 이웃

K-최근접 이웃(K-Nearest Neighbor, KNN) 알고리즘은 새로운 입력을 받을 때, 기존 클러스터에서 모든 데이터와 인스턴스 기반 거리를 측정한 후 가장 많은 속성을 가진 클러스터에 할당하는 분류 알고리즘임

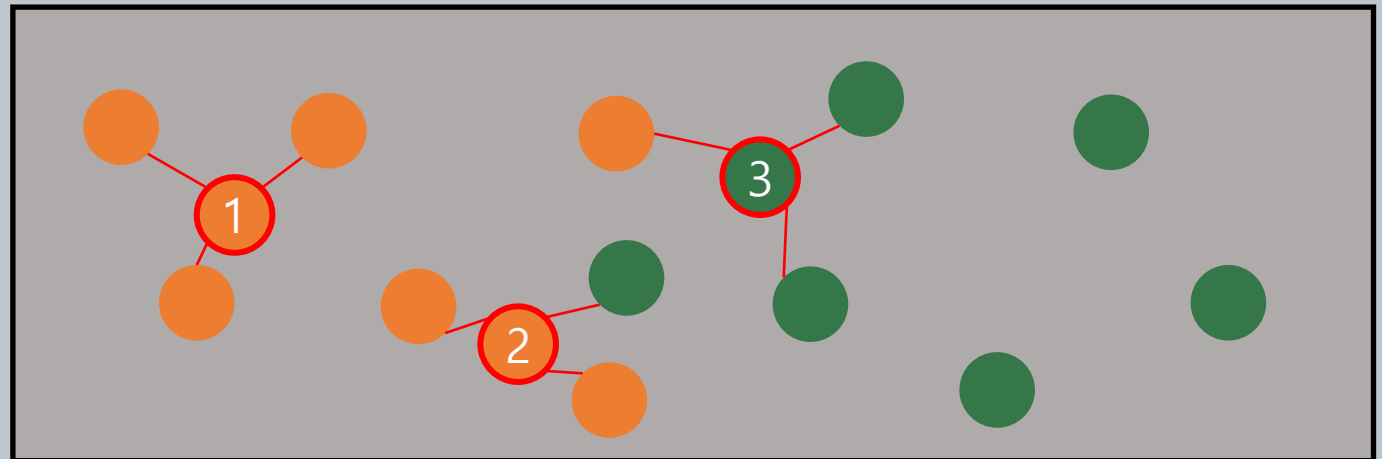
과거 데이터를 저장해두고, 새로운 데이터가 추가 및 삭제될 때마다 비교를 수행하는 방식임

입력 1: 주변 범주 세 개가 주황색이므로 주황색 분류

입력 2: 주변 범주 두 개가 주황색, 한 개가 녹색이므로 주황색으로 분류

입력 3: 주변 범주 두 개가 녹색, 한 개가 주황색으로 녹색으로 분류

K: 5



지도 학습 알고리즘: K-최근접 이웃

KNN 알고리즘 학습을 수행하기 위해 데이터 호출 및 모델 생성, 이후 K 값 예측을 수행함

적정한 K 값에 대한 예측

라이브러리 호출

numpy, matplotlib, pandas...

데이터세트 로딩

iris.data

데이터세트 분리

Train data : Test data = 8 : 2

모델 생성

KNeighborsClassifier()

K 값 예측

predict {"accuracy", "num_k"}

지도 학습 알고리즘: K-최근접 이웃

KNN 모델 학습

Import Library

```
import os
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

```
# 연산 처리
# 데이터 시각화
# 데이터 분석
```

```
# 데이터 세트 분리
# 데이터 특성 스케일링 -> mean 0, std 1
# k-nn 모델
# 성능 평가 지표
```

지도 학습 알고리즘: K-최근접 이웃

KNN 모델 학습

Dataset check

```
names = ["sepal-length", "sepal-width", "petal-length", "petal-width", "Class"]
```

```
data_path = os.path.join("data", "iris.data")
```

```
dataset = pd.read_csv(data_path, names=names)
```

```
print(dataset.head())
```

	sepal-length	sepal-width	petal-length	petal-width	Class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

지도 학습 알고리즘: K-최근접 이웃

KNN 모델 학습

```
# Data split
x = dataset.iloc[:, :-1].values
y = dataset.iloc[:, 4].values

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.20)

s = StandardScaler()

s.fit(x_train)                                # StandardScaler 사용을 위해 데이터 맞춤
s.fit(x_test)

x_train = s.transform(x_train)
x_test = s.transform(x_test)
```


지도 학습 알고리즘: K-최근접 이웃

KNN 모델 학습

```
# Model training
knn = KNeighborsClassifier(n_neighbors=50)
knn.fit(x_train, y_train)

# Evaluate metrics
y_pred = knn.predict(x_test)
print(f"정확도: {accuracy_score(y_test, y_pred)}")
```

정확도: 0.8333333333333334

지도 학습 알고리즘: K-최근접 이웃

KNN 알고리즘의 최적의 k 값 탐색 (1 ~ 10)

```
k = 10
```

```
acc_array = np.zeros(k)
```

```
for k in np.arange(1, k+1, 1):  
    classifier = KNeighborsClassifier(n_neighbors=k).fit(x_train, y_train)  
    y_pred = classifier.predict(x_test)  
    acc = accuracy_score(y_test, y_pred)  
    acc_array[k-1] = acc
```

```
max_acc = np.amax(acc_array)  
acc_list = list(acc_array)  
k = acc_list.index(max_acc)
```

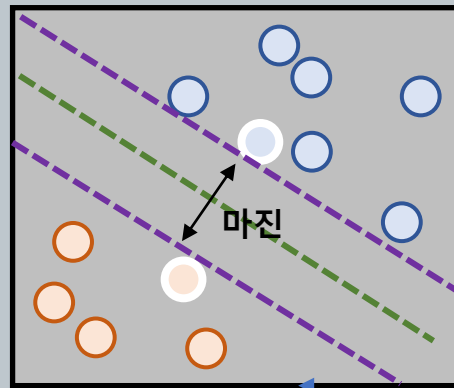
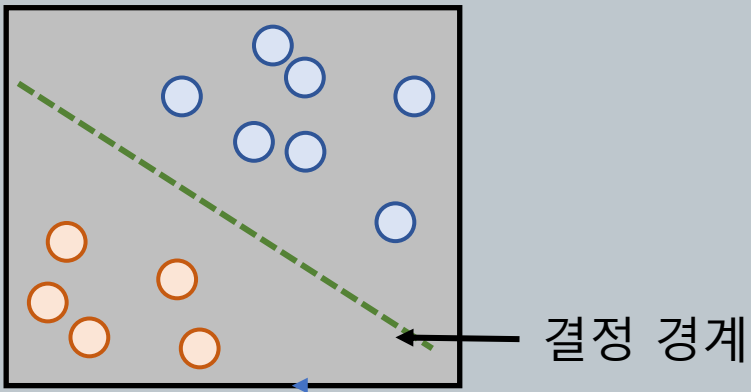
```
print(f"정확도 {max_acc}으로 최적의 k는 {k+1} 입니다.")
```

정확도 1.0으로 최적의 k는 8 입니다.

지도 학습 알고리즘: 서포트 벡터 머신

서포트 벡터 머신(Support Vector Machine, SVM)은 분류를 위한 기준선을 정의하는 모델로, 분류되지 않은 새로운 데이터를 분류하기 위한 결정 경계(기준선)를 결정함

결정 경계는 데이터가 분류된 클래스에서 최대한 멀리 떨어져 있을 때 성능이 가장 좋으며, 이는 데이터 클러스터 간의 마진(Margin)을 최대화하는 것과 같음

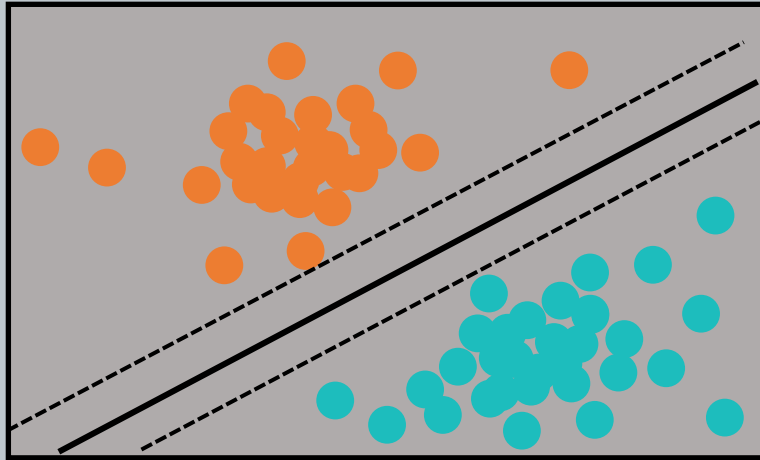


마진: 결정 경계와 서포트 벡터 사이의 거리

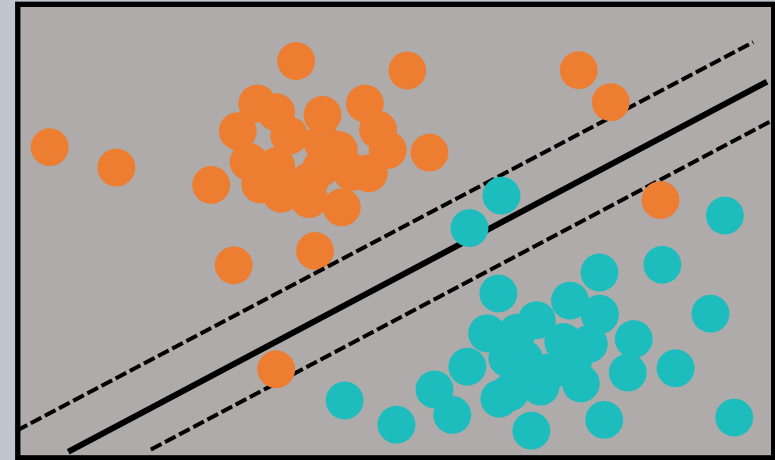
서포트 벡터: 결정 경계와 가까이 있는 데이터
(흰색 테두리 데이터)

지도 학습 알고리즘: 서포트 벡터 머신

서포트 벡터 머신은 데이터를 올바르게 분리하면서 마진의 크기를 최대화해야 하기 때문에, 이상치(Outlier) 데이터 제어가 중요하며, 이상치를 허용하지 않는 방식을 하드 마진(Hard Margin)이라 하며, 이상치를 허용하는 방식을 소프트 마진(Soft Margin)이라 함



하드 마진 SVM



소프트 마진 SVM

지도 학습 알고리즘: 서포트 벡터 머신

서포트 벡터 머신 모델 학습

```
import os
```

```
from sklearn import svm  
from sklearn import datasets  
from sklearn.metrics import accuracy_score  
from sklearn.model_selection import train_test_split
```

```
os.environ["TF_CPP_MIN_LOG_LEVEL"] = '3'          # 로깅을 제어 {0: 기본 값, 1: INFO, 2: WARNING, 3:  
ERROR}
```

지도 학습 알고리즘: 서포트 벡터 머신

서포트 벡터 머신 모델 학습

```
iris = datasets.load_iris()
x_train, x_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.6, random_state=42
)

svm = svm.SVC(kernel="linear", C=1.0, gamma=0.5)
svm.fit(x_train, y_train)

predict = svm.predict(x_test)
score = accuracy_score(y_test, predict)

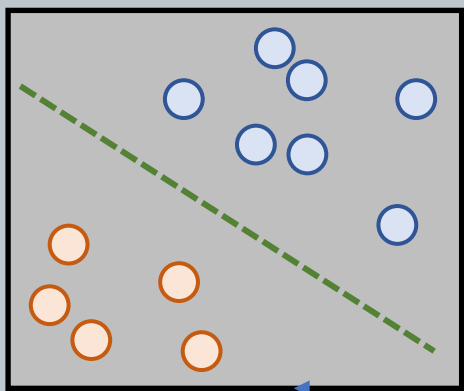
print(f"정확도: {score:f}")
```

정확도: 0.988889

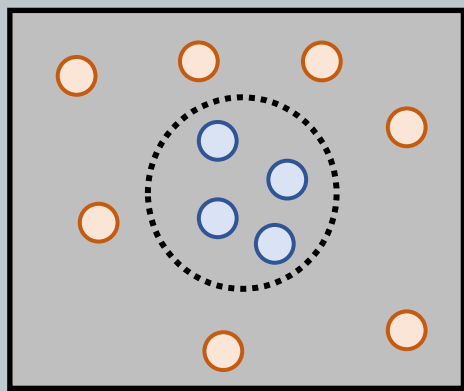
지도 학습 알고리즘: 서포트 벡터 머신

SVM은 선형 분류와 비선형 분류를 지원하며, 비선형 커널은 선형 분류를 수행할 수 없는 데이터를 해결할 수 있음

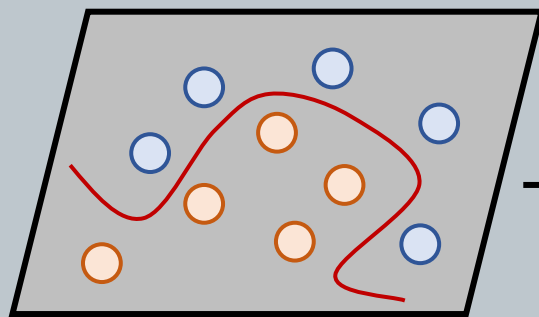
비선형 문제를 해결하기 위한 기본적인 방법은 저차원의 데이터를 고차원으로 전달하는 것이지만, 연산량 증가 문제가 발생하기 때문에 SVM에서는 커널 트릭(Kernel Trick) 기법을 활용하여 비선형 문제를 해결함



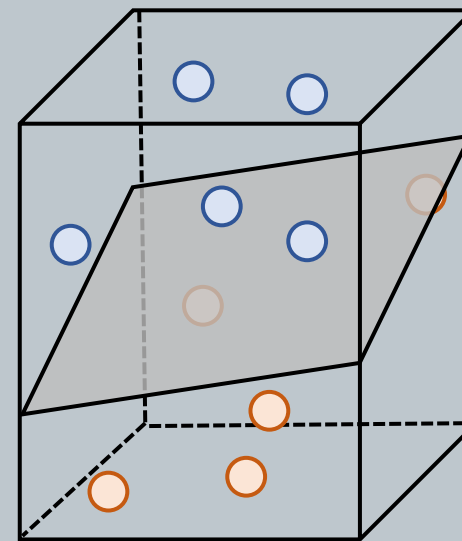
선형 분류



비선형 분류



저차원 데이터



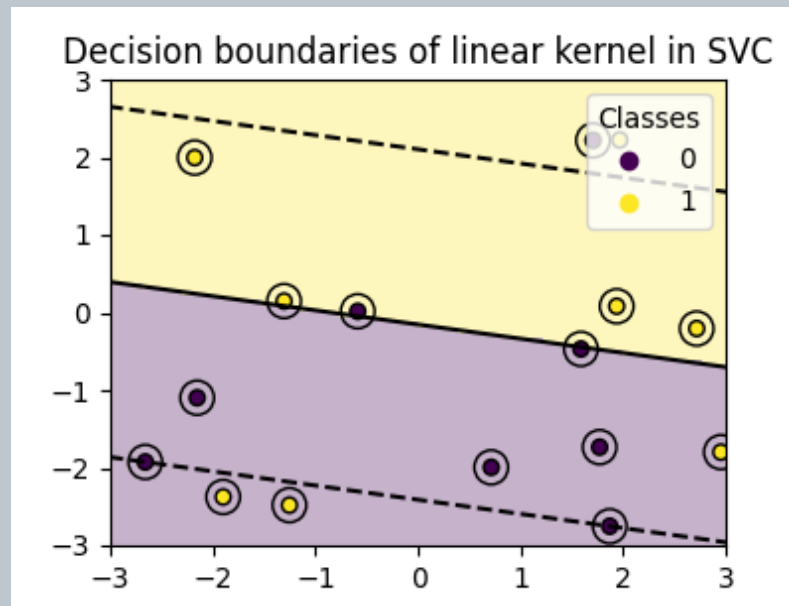
고차원 데이터

지도 학습 알고리즘: 서포트 벡터 머신

커널을 사용하는 방식에는 선형과 비선형으로 분류할 수 있고, 선형 모델을 위한 커널에는 선형(Linear) 커널이 있으며, 비선형 모델을 위한 커널에는 RBF(Radial Basis Function) 커널과 다항식(Poly) 커널이 존재함

선형 커널은 선형으로 분류 가능한 데이터에 적용하며, 다음처럼 계산되며, 커널 트릭 없이 선형 분류를 수행함

$$K(x_1, x_2) = x_1^T \cdot x_2$$

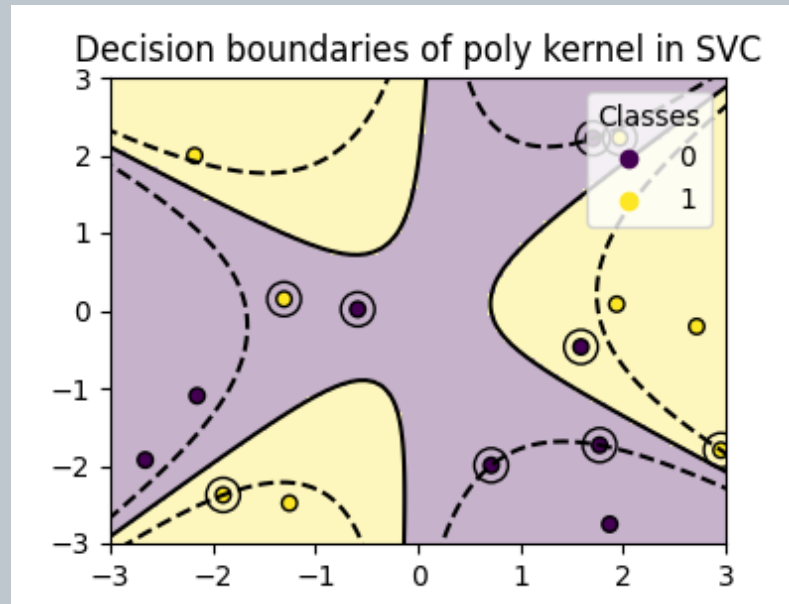


지도 학습 알고리즘: 서포트 벡터 머신

다항식 커널은 실제 특성을 추가하지 않지만, 추가한 것과 같은 결과를 얻을 수 있는 방법으로, 고차원 데이터 매핑이 가능하며, 다음처럼 계산됨

$$K(x_1, x_2) = (\gamma x_1^T \cdot x_2 + r)^d$$

(γ : 결정 계수, r : 편향(bias), d : 차원)

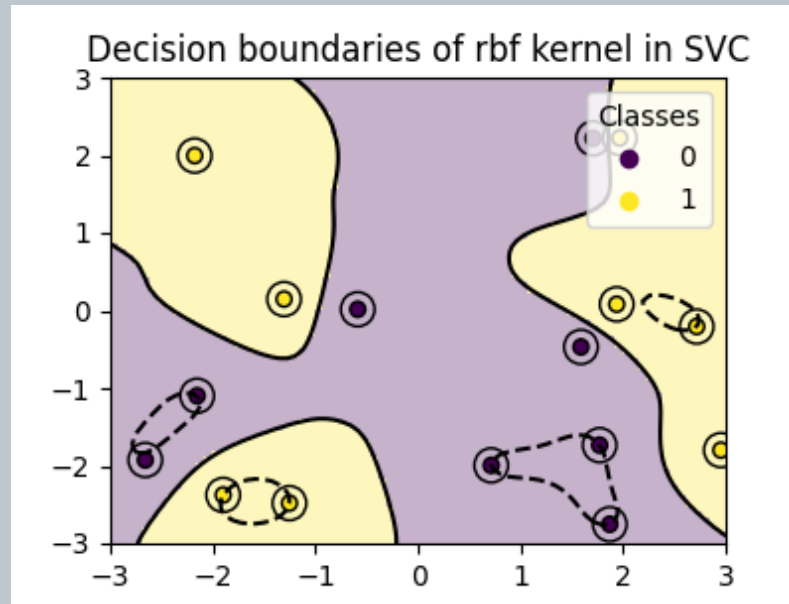


지도 학습 알고리즘: 서포트 벡터 머신

RBF 커널은 다항식 커널의 확장 형태로, 가우시안 커널로도 알려져 있으며, 모든 차수의 모든 다항식을 고려하여 입력 벡터를 무한한 고차원으로 매핑함

다항식 커널에는 차수 입력에 한계가 있지만, RBF 커널에는 차수에 제한이 없음

$$K(x_1, x_2) = \exp(-\gamma \|a - b\|^2)$$

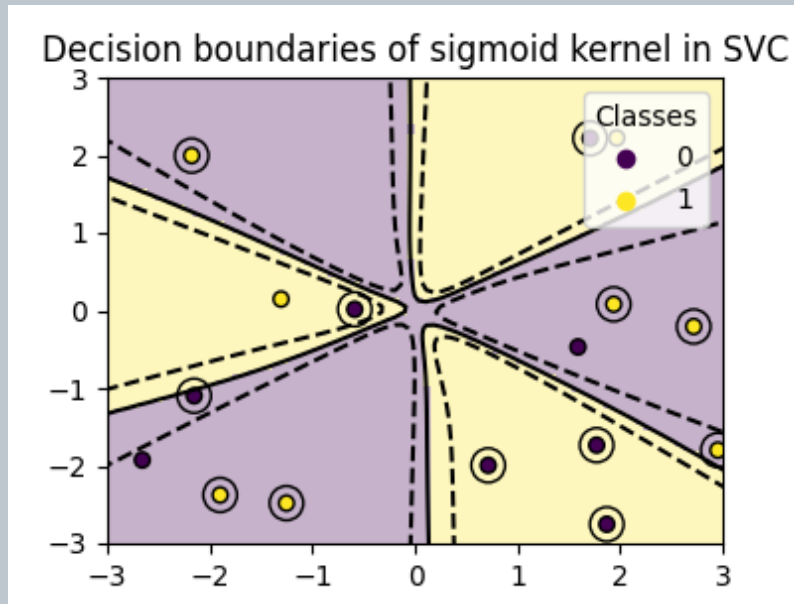


지도 학습 알고리즘: 서포트 벡터 머신

추가로, 시그모이드(Sigmoid) 커널을 활용할 수도 있는데, 이 커널은 시그모이드 곡선에 맞춰서 클래스를 분리하도록 계산되기 때문에 일반화 성능이 떨어짐

대신, 시그모이드 형태로 분류되는 데이터를 처리할 때 효과적으로 사용할 수 있기 때문에 사용 빈도가 낮음

$$K(x_1, x_2) = \tanh(\gamma \cdot x_1^T x_2 + r)$$

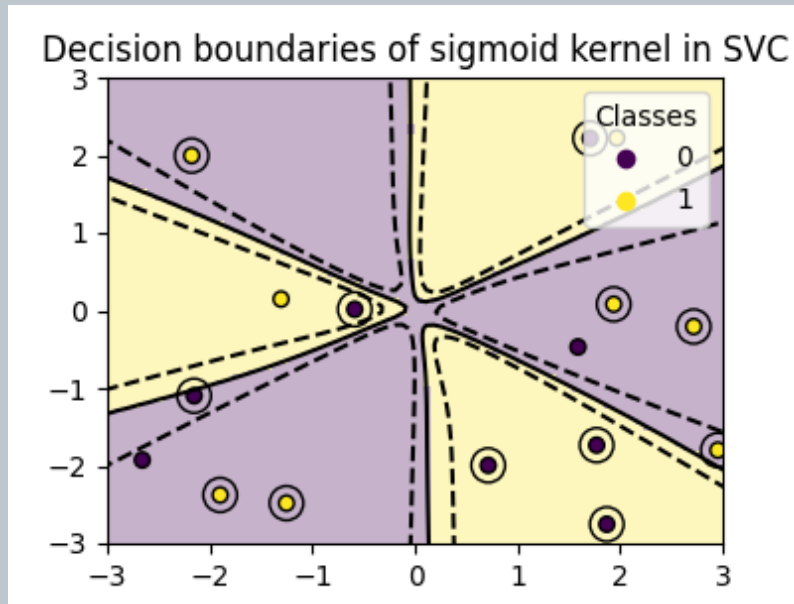


지도 학습 알고리즘: 서포트 벡터 머신

추가로, 시그모이드(Sigmoid) 커널을 활용할 수도 있는데, 이 커널은 시그모이드 곡선에 맞춰서 클래스를 분리하도록 계산되기 때문에 일반화 성능이 떨어짐

대신, 시그모이드 형태로 분류되는 데이터를 처리할 때 효과적으로 사용할 수 있기 때문에 사용 빈도가 낮음

$$K(x_1, x_2) = \tanh(\gamma \cdot x_1^T x_2 + r)$$



지도 학습 알고리즘: 서포트 벡터 머신

SVC 커널 비교

```
import random
import matplotlib.pyplot as plt
import numpy as np

from sklearn import svm
from sklearn.inspection import DecisionBoundaryDisplay

data = np.array([
    [random.uniform(-3.0, 3.0), random.uniform(-3.0, 3.0)]
    for _ in range(0, 16)
])

label = np.array([
    0, 0, 0, 0, 0, 0, 0, 0,
    1, 1, 1, 1, 1, 1, 1, 1
])

fig, ax = plt.subplots(figsize=(4, 3))
x_min, x_max, y_min, y_max = -3, 3, -3, 3
scatter = ax.scatter(data[:, 0], data[:, 1], s=150, c=label, label=label, edgecolors='k')
ax.legend(*scatter.legend_elements(), loc="upper right", title="Classes")
ax.set_title("Sample in two-dimensional feature space")
plt.show()

plot_training_data_with_decision_boundary(x=data, y=label, kernel="linear")
plot_training_data_with_decision_boundary(x=data, y=label, kernel="poly")
plot_training_data_with_decision_boundary(x=data, y=label, kernel="rbf")
plot_training_data_with_decision_boundary(x=data, y=label, kernel="sigmoid")
```

지도 학습 알고리즘: 서포트 벡터 머신

SVC 커널 비교 (plot_training_data_with_decision_boundary)

```
def plot_training_data_with_decision_boundary(
    x, y, kernel, ax=None, long_title=True, support_vectors=True
):
    model = svm.SVC(kernel=kernel, gamma=2).fit(x, y)

    if ax is None:
        _, ax = plt.subplots(figsize=(4, 3))

    x_min, x_max, y_min, y_max = -3, 3, -3, 3
    ax.set(xlim=(x_min, x_max), ylim=(y_min, y_max))

    common_params = {"estimator": model, 'X': x, "ax": ax}
```

지도 학습 알고리즘: 서포트 벡터 머신

SVC 커널 비교 (plot_training_data_with_decision_boundary)

```
DecisionBoundaryDisplay.from_estimator(  
    **common_params,  
    response_method="predict",  
    plot_method="pcolormesh",  
    alpha=0.3  
)  
DecisionBoundaryDisplay.from_estimator(  
    **common_params,  
    response_method="decision_function",  
    plot_method="contour",  
    levels=[-1, 0, 1],  
    colors=['k', 'k', 'k'],  
    linestyles=["--", "-", "--"]  
)  
if support_vectors:  
    ax.scatter(  
        model.support_vectors_[0],  
        model.support_vectors_[1],  
        s=150,  
        facecolors="none",  
        edgecolors='k'  
    )
```

지도 학습 알고리즘: 서포트 벡터 머신

SVC 커널 비교 (plot_training_data_with_decision_boundary)

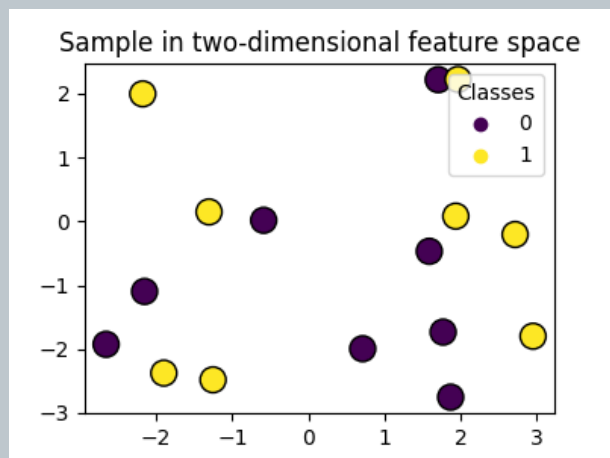
```
ax.scatter(x[:, 0], x[:, 1], c=y, s=30, edgecolors='k')
ax.legend(*scatter.legend_elements(), loc="upper right", title="Classes")

if long_title:
    ax.set_title(f"Decision boundaries of {kernel} kernel in SVC")
else:
    ax.set_title(kernel)

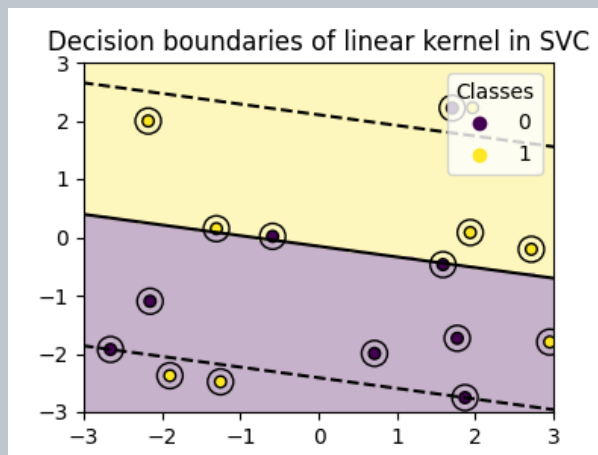
plt.show()
```


지도 학습 알고리즘: 서포트 벡터 머신

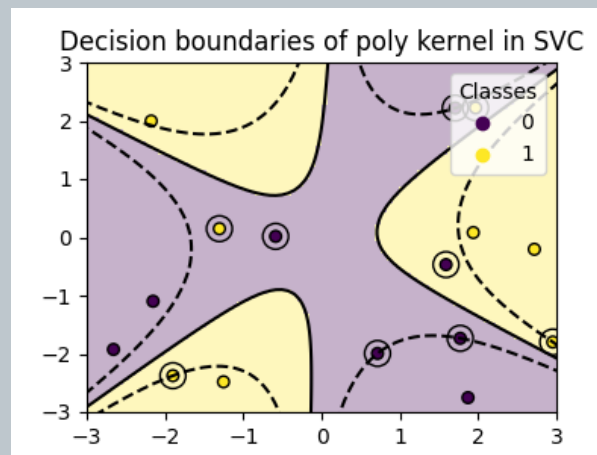
SVC 커널 비교 결과



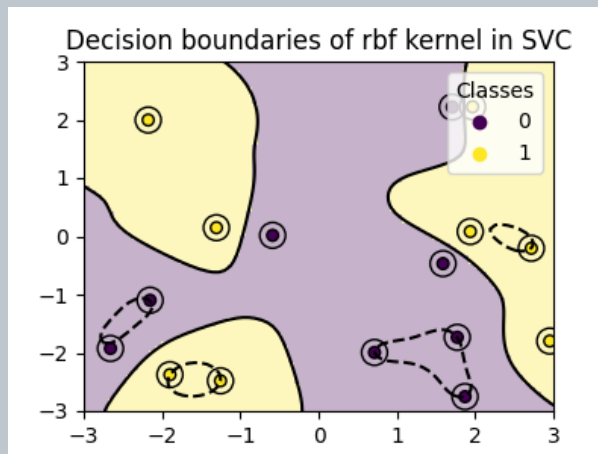
입력 데이터



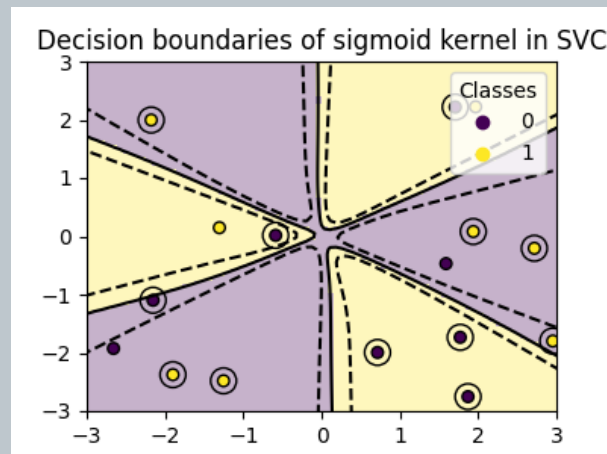
선형 커널



다항식 커널



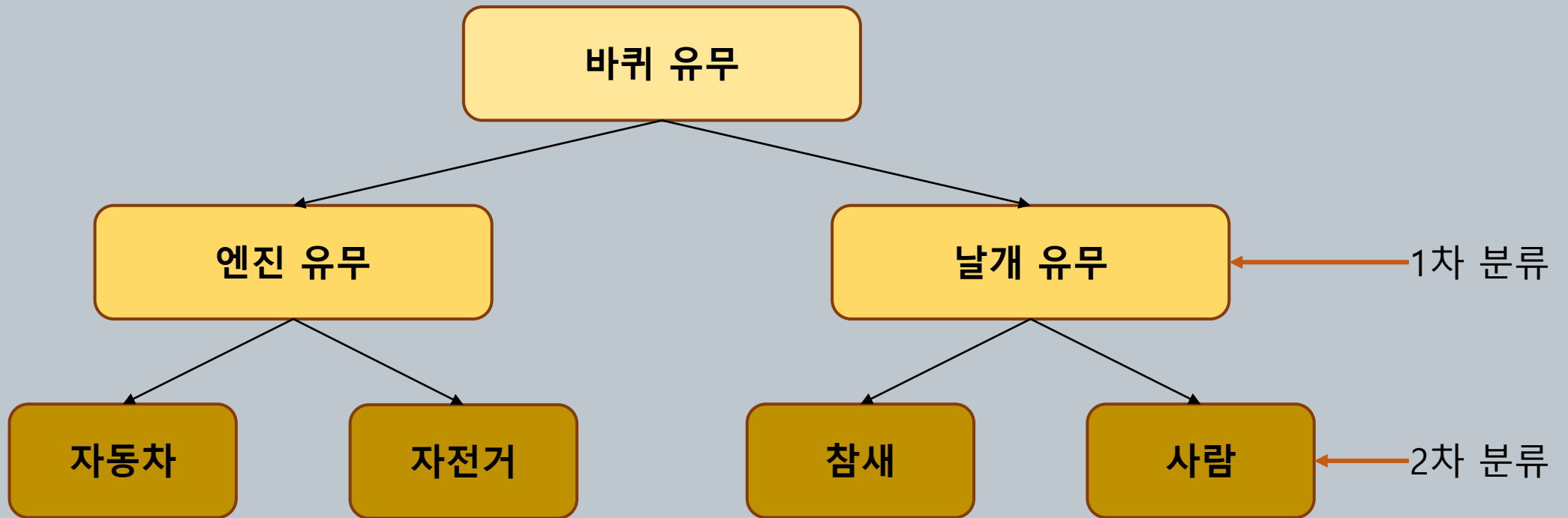
rbf 커널



시그모이드 커널

지도 학습 알고리즘: 결정 트리

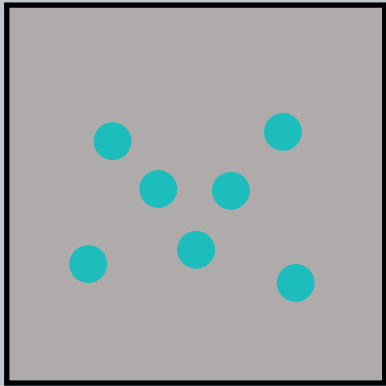
결정 트리(Decision Tree) 알고리즘은 데이터를 분류하거나 결과 값을 예측하는 분석 방법으로, 결과 모델이 트리 구조로 나타나기 때문에 결정 트리라고 불림



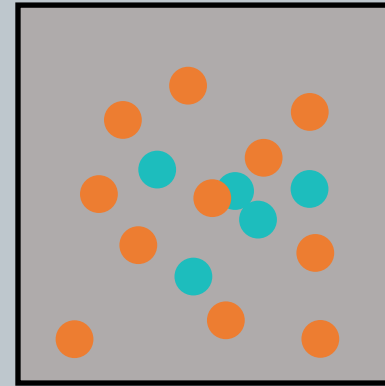
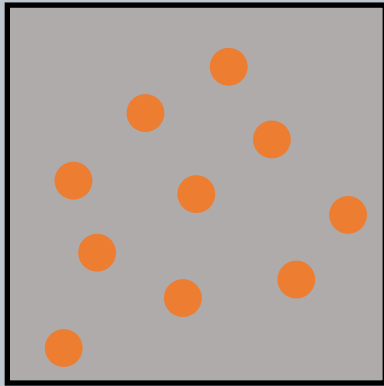
지도 학습 알고리즘: 결정 트리

결정 트리에서 1차 분류 이후 각 영역의 순도를 증가시키고, 불순도와 불확실성을 감소하는 방향으로 학습하며, 이를 정보 획득(Information Gain)이라 함

순도	범주 안에서 같은 종류의 데이터만 모여 있는 상태
불순도	서로 다른 데이터가 섞여 있는 상태



순도 100% 상태



불순도 상태

지도 학습 알고리즘: 결정 트리

결정 트리에서 불확실성을 계산하는 방법으로는 엔트로피와 지니 계수 방식을 통해 불확실성과 불순도를 측정함

엔트로피(Entropy)는 확률 변수의 불확실성을 수치로 나타낸 방식으로, 엔트로피가 높을수록 불확실성이 높다는 의미임

$Entropy_{data} = \{0, 0.5\}$ 상황에서, 엔트로피가 0인 경우가 불확실성 최소, 순도 최대, 0.5인 경우가 불확실성 최대, 순도 최소인 상태를 의미함

$$Entropy(A) = \sum_{k=1}^m p_k \log_2(p_k)$$

($p_k = A$: 영역에 속하는 데이터 중에서 k 범주에 속하는 데이터 비율)

동전이 앞면이 나올 확률 $\frac{1}{4}$ 와 뒷면이 나올 확률 $\frac{3}{4}$ 일 때, 엔트로피를 계산하면

$$\begin{aligned} Entropy(A) &= -\left(\frac{1}{4}\right) \log\left(\frac{1}{4}\right) - \left(\frac{3}{4}\right) \log\left(\frac{3}{4}\right) \\ &= \frac{2}{4} + \frac{3}{4} \times 0.42 \\ &= 0.31 \end{aligned}$$

지도 학습 알고리즘: 결정 트리

지니 계수(Gini Index)는 불순도를 측정하는 지표로, 데이터의 통계적 분산 정도를 정량화해서 표현한 값임

$$G(S) = 1 - \sum_{i=1}^c p_i^2$$

(S : 이미 발생한 사건의 모음, c : 사건 개수)

지니 계수가 높을 수록 데이터가 분산되어 있음을 의미함

또한, log 계산이 포함되어 있지 않기 때문에, 엔트로피 방식보다 계산 속도가 빨라서 결정 트리에서 많이 사용됨

지도 학습 알고리즘: 결정 트리

결정 트리 모델 학습

```
import os
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

data_path = os.path.join("data", "train.csv")
df = pd.read_csv(data_path, index_col="PassengerId")

print(df.head())
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

지도 학습 알고리즘: 결정 트리

결정 트리 모델 학습

```
df = df[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Survived']]
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df = df.dropna()
x = df.drop('Survived', axis=1)
y = df['Survived']

x_train, x_test, y_train, y_test = train_test_split(x, y, random_state=1)

model = DecisionTreeClassifier()
model.fit(x_train, y_train)

y_predict = model.predict(x_test)
score = accuracy_score(y_test, y_predict)

print(f"정확도: {score:f}")
```

정확도: 0.821229

지도 학습 알고리즘: 결정 트리

결정 트리 모델 학습

```
confused_df = pd.DataFrame(  
    confusion_matrix(y_test, y_predict),  
    columns=['Predicted Not Survival', 'Predicted Survival'],  
    index=['True Not Survival', 'True Survival']  
)  
  
print(confused_df)
```

	Predicted Not Survival	Predicted Survival
True Not Survival	98	14
True Survival	18	49

지도 학습 알고리즘: 로지스틱 회귀

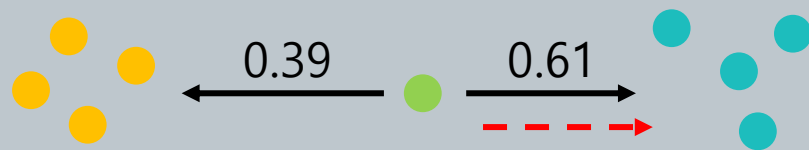
회귀(Regression)는 변수가 두 개 주어졌을 때, 한 변수에서 다른 변수를 예측하거나 두 변수의 관계를 규명하는 데 사용하는 방법으로, 독립 변수와 종속 변수를 사용할 수 있음

독립 변수(예측 변수)	영향을 미칠 것으로 예상되는 변수
종속 변수(기준 변수)	영향을 받을 것으로 예상되는 변수

회귀에 주어진 두 변수에 대하여, 독립 변수와 종속 변수의 설정은 논리적인 타당성이 있어야 함
(ex : 몸무게(종속 변수) – 키(독립 변수))

지도 학습 알고리즘: 로지스틱 회귀

로지스틱 회귀(Logistic Regression)는 분석하고자 하는 대상이 두 개 이상의 집단으로 나누어진 경우, 개별 관측치들이 어느 집단으로 분류될 수 있는지 분석하고 예측하는 모델을 개발하는 데 사용되는 통계 기법임

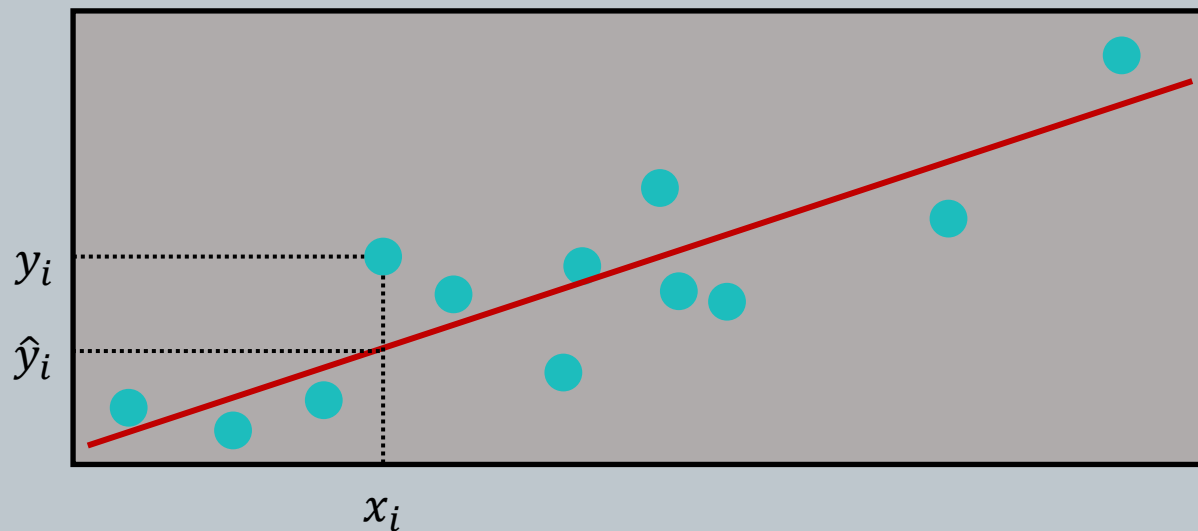


구분	일반적인 회귀 분석	로지스틱 회귀 분석
종속 변수	연속형 변수	이산형 변수
모델 탐색 방법	최소제곱법	최대우도법
모델 검정	F -테스트, t -테스트	χ^2 테스트

지도 학습 알고리즘: 로지스틱 회귀

최소제곱법(Mean Squared)은 평균 제곱 오차(Mean Square Error, MSE)와 같은 의미로, 입력 데이터 x_i 에 대하여 실제 값 y_i 에서 예측 값 $\hat{y}_i$ 를 뺀 후 제곱해서 계산되는 값으로, 일반 회귀 분석에서 사용됨

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



지도 학습 알고리즘: 로지스틱 회귀

우도(Likelihood)는 나타난 결과에 따라 여러 가능한 가설을 평가할 수 있는 척도를 의미하며, 최대우도법(Maximum Likelihood)은 나타난 결과에 해당하는 가설마다 계산된 우도 값 중에서 가장 큰 값을 계산하는 방식임

$$\theta_{ml} = \operatorname{argmax}_{\theta} P_{model}(Y|X; \theta)$$
$$\theta_{ml} = \operatorname{argmax}_{\theta} \sum_{i=1}^m \log P_{model}(y_i|X; \theta)$$

입력 X 와 모델 파라미터 θ 가 주어질 때, Y 가 나타날 확률을 최대화하는 θ 를 찾는 방식으로, X 와 Y 가 고정된 상태에서 θ 를 조절하여 최적의 θ_{ml} 을 탐색함

최대우도의 추정치를 계산하기 위해 두 번째 식을 사용함

지도 학습 알고리즘: 로지스틱 회귀

로지스틱 회귀 모델 학습

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix

digits = load_digits()

print(f"Image Data Shape: {digits.data.shape}")
print(f"Label Data Shape: {digits.target.shape}")
```

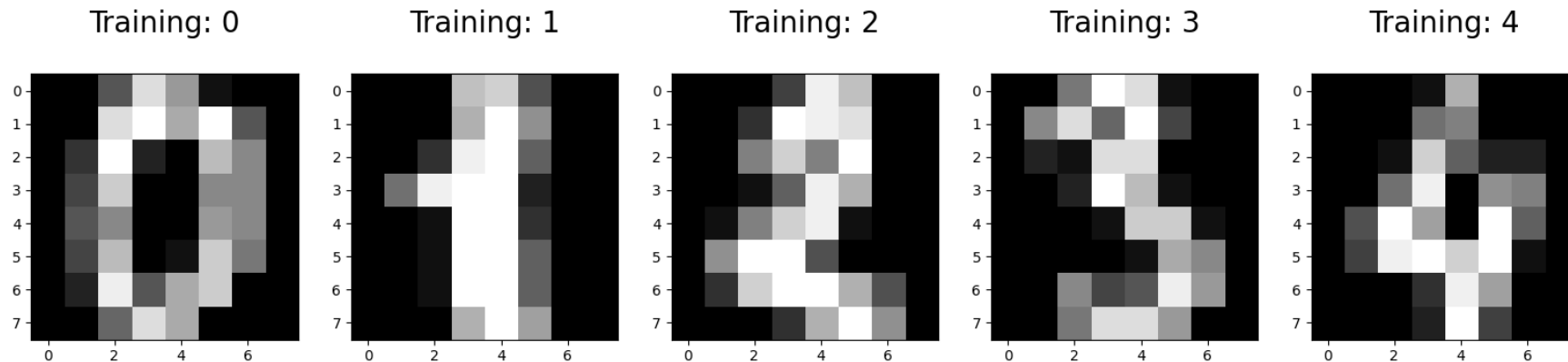
```
Image Data Shape: (1797, 64)
Label Data Shape: (1797,)
```

지도 학습 알고리즘: 로지스틱 회귀

로지스틱 회귀 모델 학습

```
plt.figure(figsize=(20,4))
```

```
for index, (image, label) in enumerate(zip(digits.data[0:5], digits.target[0:5])):  
    plt.subplot(1, 5, index + 1)  
    plt.imshow(np.reshape(image, (8,8)), cmap=plt.cm.gray)  
    plt.title('Training: %i\n' % label, fontsize = 20)
```



지도 학습 알고리즘: 로지스틱 회귀

로지스틱 회귀 모델 학습

```
x_train, x_test, y_train, y_test = train_test_split(digits.data, digits.target, test_size=0.25,
random_state=0)
```

```
logistic_regression = LogisticRegression(max_iter=100)
logistic_regression.fit(x_train, y_train)
```

```
print(logistic_regression.predict(x_test[0].reshape(1, -1)))
print(logistic_regression.predict(x_test[0:10]))
```

```
y_pred = logistic_regression.predict(x_test)
score = logistic_regression.score(x_test, y_test)
```

```
print(f"Performance: {score}")
```

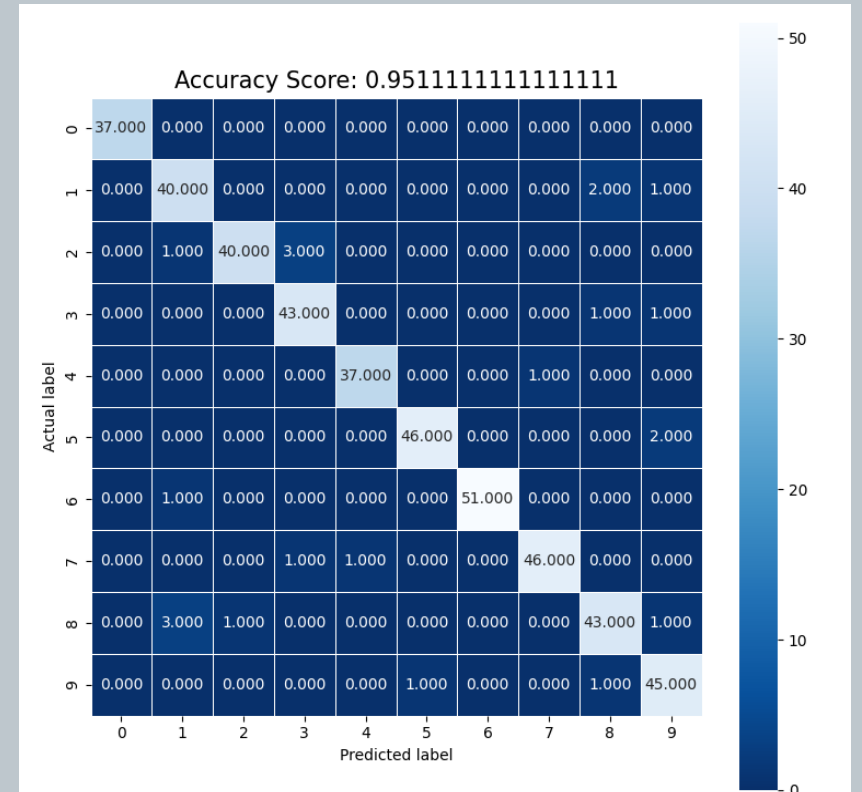
```
[2]
[2 8 2 6 6 7 1 9 8 5]
Performance: 0.9511111111111111
```

지도 학습 알고리즘: 로지스틱 회귀

로지스틱 회귀 모델 학습

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(9,9))
sns.heatmap(cm, annot=True, fmt=".3f", linewidths=.5, square = True, cmap = 'Blues_r');
plt.ylabel('Actual label');
plt.xlabel('Predicted label');
all_sample_title = 'Accuracy Score: {0}'.format(score)
plt.title(all_sample_title, size = 15);
plt.show();
```

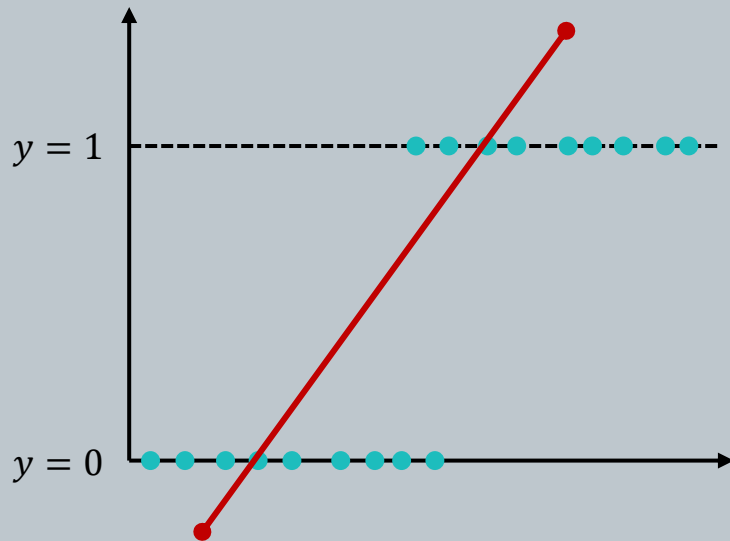


지도 학습 알고리즘: 선형 회귀

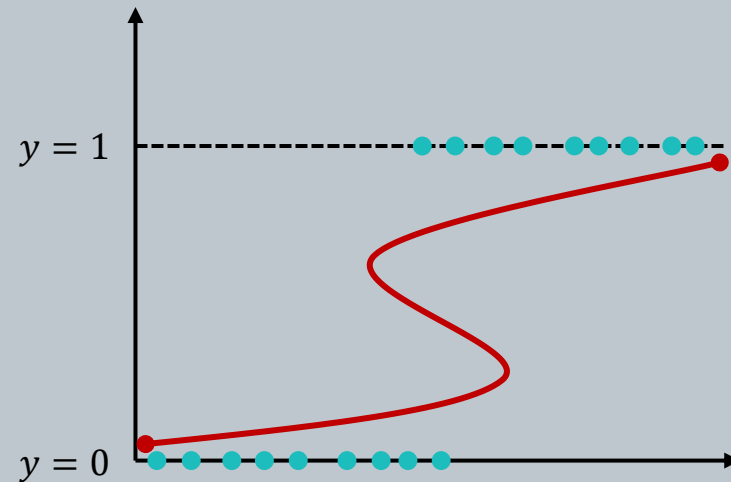
선형 회귀(Linear Regression)은 독립 변수 x 를 사용하여 종속 변수 y 의 움직임을 예측하고 설명하는 데 사용되며, 독립 변수 x 는 하나일 수도 있고 $x_i \in X$ 처럼 여러 개일 수도 있음

선형 회귀는 종속 변수와 독립 변수의 관계를 설정하는 데 사용되며, 로지스틱 회귀는 사건의 확률을 확인하는 데 사용됨

$x \rightarrow y$: 단순 선형 회귀
 $x_i \in X \rightarrow y$: 다중 선형 회귀



선형 회귀



로지스틱 회귀

지도 학습 알고리즘: 선형 회귀

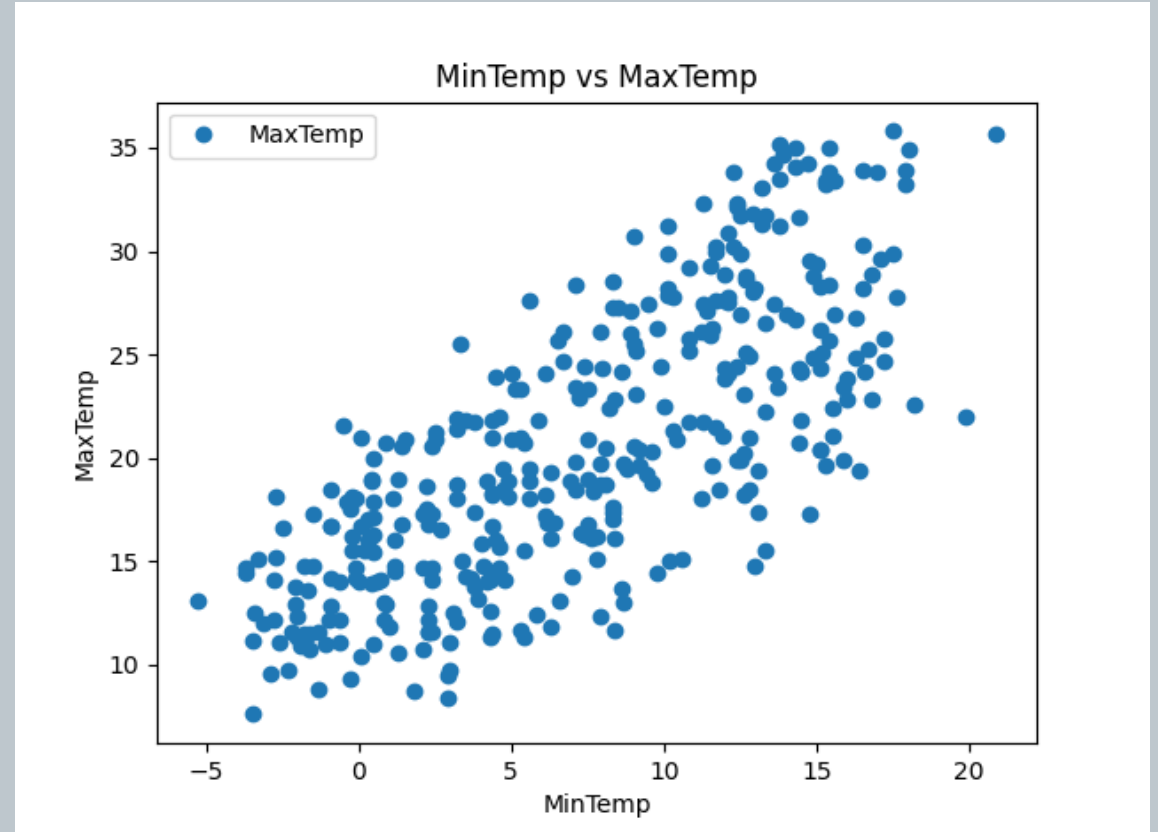
선형 회귀 모델 학습

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as seabornInstance

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

data_path = os.path.join("data", "weather.csv")
dataset = pd.read_csv(data_path)

dataset.plot(x='MinTemp', y='MaxTemp', style='o')
plt.title('MinTemp vs MaxTemp')
plt.xlabel('MinTemp')
plt.ylabel('MaxTemp')
plt.show()
```



지도 학습 알고리즘: 선형 회귀

선형 회귀 모델 학습

```
x = dataset['MinTemp'].values.reshape(-1,1)
y = dataset['MaxTemp'].values.reshape(-1,1)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)

regressor = LinearRegression()
regressor.fit(x_train, y_train)

y_pred = regressor.predict(x_test)

df = pd.DataFrame({'Actual': y_test.flatten(), 'Predicted': y_pred.flatten()})
print(df)
```

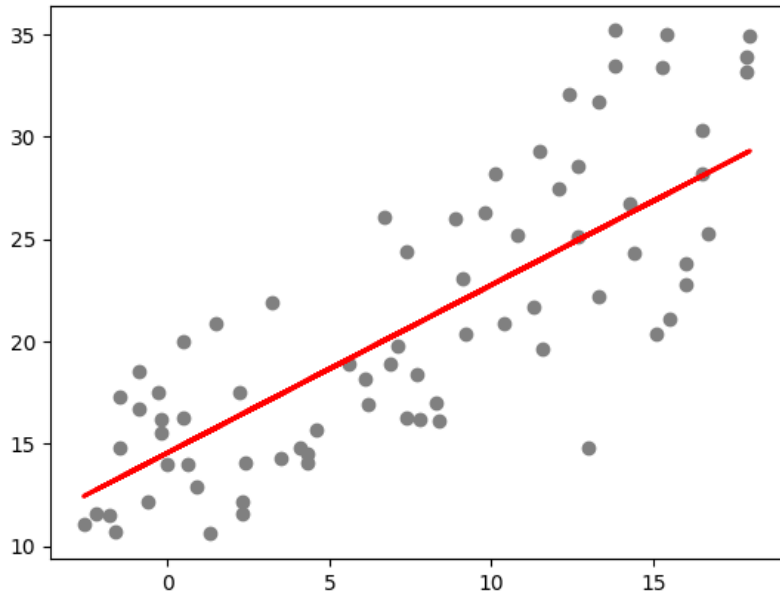
	Actual	Predicted
0	25.2	23.413030
1	11.5	13.086857
2	21.1	27.264856
3	22.2	25.461874
4	20.4	26.937041

지도 학습 알고리즘: 선형 회귀

선형 회귀 모델 학습

```
plt.scatter(x_test, y_test, color='gray')
plt.plot(x_test, y_pred, color='red', linewidth=2)
plt.show()

print('평균제곱법:', mean_squared_error(y_test, y_pred))
print('루트 평균제곱법:', np.sqrt(mean_squared_error(y_test, y_pred)))
```



평균제곱법: 17.011877668640622
루트 평균제곱법: 4.124545753006096